

Automatic Differentiation and Differentiable Programming

Simulation Intelligence

Automatic Differentiation (AD)

To solve optimisation problems using gradient methods we need to compute the gradients (derivatives) of the objective with respect to the parameters.

To measure how much the model output is robust to perturbations in the input we need to compute the derivatives of the output with respect to the input.

Idea of **differentiable programming**.

Automatic Differentiation (AD)

Differentiable programming

Consider a neural network $f_\theta : X \rightarrow Y$, with a loss function $\mathcal{L} : Y \times Y \rightarrow \mathbb{R}$, s.t.
 $\mathcal{L}(y, f_\theta(\mathbf{x})) \in \mathbb{R}$ measures the error made by the neural network in predicting y for input $\mathbf{x}$.

We may need to compute the gradients of the loss with respect to the parameters θ :

$$\nabla_\theta \mathcal{L}(\theta) = \left[\frac{\partial \mathcal{L}}{\partial \theta_1}(\theta), \dots, \frac{\partial \mathcal{L}}{\partial \theta_{|\Theta|}}(\theta) \right].$$

We may also need to compute the gradients of f_θ with respect to the input x :

$$\nabla_x f_\theta(\mathbf{x}) = \left[\frac{\partial f_\theta}{\partial x_1}(\mathbf{x}), \dots, \frac{\partial f_\theta}{\partial x_n}(\mathbf{x}) \right],$$

assuming $Y = \mathbb{R}$ otherwise we have the Jacobian matrix.

Compute derivatives...

There are three ways to compute derivatives:

1. **Symbolically** differentiate the function wrt its parameters by hand using a computer algebra system (CAS)

Problem: Static - can't "differentiate algorithms"

2. Make estimates using **Finite Differences**

Problem: Numerical errors - will compound in deep nets

3. Use **Automatic Differentiation**

Automatic Differentiation (AD)

Automatic Differentiation is:

a method to get **exact** derivatives **efficiently**, by storing information as you go forward that you can reuse as you go backwards.

- Takes code that computes a function and uses that to compute the derivative of that function.
- The goal isn't to obtain closed-form solutions, but to be able to write a program that efficiently computes the derivatives.

Chain rule for differentiation

We need to formalise a set of rules for translating a program that evaluates an expression into a program that evaluates its derivatives.

The chain rule for a variable/function z that depends on y which depends on x :

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

In general, when w is some output variable, and u_i denotes each input variable w depends on:

$$\frac{\partial w}{\partial t} = \sum_{i=1}^N \frac{\partial w}{\partial u_i} \frac{\partial u_i}{\partial t}$$

Chain rule — Example

Chain rule — Example

Example (Math)

$x = ?$

$y = ?$

$a = x y$

$b = \sin(x)$

$z = a + b$

Example (Code)

```
x = ?
```

```
y = ?
```

```
a = x * y
```

```
b = sin(x)
```

```
z = a + b
```

Chain rule — Example

Example (Math)

$$\begin{aligned}x &=? \\ y &=? \\ a &= x y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

Example (Code)

```
x = ?  
y = ?  
a = x * y  
b = sin(x)  
z = a + b
```

$$\begin{aligned}\frac{\partial x}{\partial t} &=? \\ \frac{\partial y}{\partial t} &=? \\ \frac{\partial a}{\partial t} &= x \frac{\partial y}{\partial t} + y \frac{\partial x}{\partial t} \\ \frac{\partial b}{\partial t} &= \cos(x) \frac{\partial x}{\partial t} \\ \frac{\partial z}{\partial t} &= \frac{\partial a}{\partial t} + \frac{\partial b}{\partial t}\end{aligned}$$

Chain rule — Example

Example (Math)

$$\begin{aligned}x &=? \\ y &=? \\ a &= x y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

Example (Code)

```
x = ?
y = ?
a = x * y
b = sin(x)
z = a + b
```

$$\frac{\partial x}{\partial t} = ?$$

$$\frac{\partial y}{\partial t} = ?$$

$$\frac{\partial a}{\partial t} = x \frac{\partial y}{\partial t} + y \frac{\partial x}{\partial t}$$

$$\frac{\partial b}{\partial t} = \cos(x) \frac{\partial x}{\partial t}$$

$$\frac{\partial z}{\partial t} = \frac{\partial a}{\partial t} + \frac{\partial b}{\partial t}$$

If we substitute $t = x$ we have an algorithm for computing $\partial z / \partial x$.

To get $\partial z / \partial y$ we'd just substitute $t = y$.

Chain rule — Example

Example (Math)

$$\begin{aligned}x &=? \\ y &=? \\ a &= x y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

Example (Code)

```
x = ?
y = ?
a = x * y
b = sin(x)
z = a + b
```

$$\begin{aligned}\frac{\partial x}{\partial t} &=? \\ \frac{\partial y}{\partial t} &=? \\ \frac{\partial a}{\partial t} &= x \frac{\partial y}{\partial t} + y \frac{\partial x}{\partial t} \\ \frac{\partial b}{\partial t} &= \cos(x) \frac{\partial x}{\partial t} \\ \frac{\partial z}{\partial t} &= \frac{\partial a}{\partial t} + \frac{\partial b}{\partial t}\end{aligned}$$

If we substitute $t = x$ we have an algorithm for computing $\partial z / \partial x$.

To get $\partial z / \partial y$ we'd just substitute $t = y$.

$$\begin{aligned}dx &= ? \\ dy &= ? \\ da &= y * dx + x * dy \\ db &= \cos(x) * dx \\ dz &= da + db\end{aligned}$$

Chain rule — Example

Example (Math)

$$\begin{aligned}x &=? \\ y &=? \\ a &= x y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

Example (Code)

```
x = ?
y = ?
a = x * y
b = sin(x)
z = a + b
```

$$\begin{aligned}\frac{\partial x}{\partial t} &=? \\ \frac{\partial y}{\partial t} &=? \\ \frac{\partial a}{\partial t} &= x \frac{\partial y}{\partial t} + y \frac{\partial x}{\partial t} \\ \frac{\partial b}{\partial t} &= \cos(x) \frac{\partial x}{\partial t} \\ \frac{\partial z}{\partial t} &= \frac{\partial a}{\partial t} + \frac{\partial b}{\partial t}\end{aligned}$$

If we substitute $t = x$ we have an algorithm for computing $\partial z / \partial x$.

To get $\partial z / \partial y$ we'd just substitute $t = y$.

```
dx = ?
dy = ?
da = y * dx + x * dy
db = cos(x) * dx
dz = da + db
```

To compute $\partial z / \partial x$ we just seed the algorithm with $dx=1$ and $dy=0$. To compute $\partial z / \partial y$ we just seed the algorithm with $dx=0$ and $dy=1$.

Chain rule — Example

Example (Math)

$$\begin{aligned}x &=? \\ y &=? \\ a &= x y \\ b &= \sin(x) \\ z &= a + b\end{aligned}$$

Example (Code)

```
x = ?
y = ?
a = x * y
b = sin(x)
z = a + b
```

We've successfully computed the gradients for a specific function, but the process was far from automatic!

$$\begin{aligned}\frac{\partial x}{\partial t} &=? \\ \frac{\partial y}{\partial t} &=? \\ \frac{\partial a}{\partial t} &= x \frac{\partial y}{\partial t} + y \frac{\partial x}{\partial t} \\ \frac{\partial b}{\partial t} &= \cos(x) \frac{\partial x}{\partial t} \\ \frac{\partial z}{\partial t} &= \frac{\partial a}{\partial t} + \frac{\partial b}{\partial t}\end{aligned}$$

If we substitute $t = x$ we have an algorithm for computing $\partial z / \partial x$.

To get $\partial z / \partial y$ we'd just substitute $t = y$.

```
dx = ?
dy = ?
da = y * dx + x * dy
db = cos(x) * dx
dz = da + db
```

To compute $\partial z / \partial x$ we just seed the algorithm with $dx=1$ and $dy=0$. To compute $\partial z / \partial y$ we just seed the algorithm with $dx=0$ and $dy=1$.

Rules

These initial rules:

$$c = a + b \quad \Rightarrow \quad dc = da + db$$

$$c = a * b \quad \Rightarrow \quad dc = b * da + a * db$$

$$c = \sin(a) \quad \Rightarrow \quad dc = \cos(a) * da$$

can easily be extended further using multivariable calculus:

$$c = a - b \quad \Rightarrow \quad dc = da - db$$

$$c = a / b \quad \Rightarrow \quad dc = da / b - a * db / b ** 2$$

$$c = a ** b \quad \Rightarrow \quad dc = b * a ** (b - 1) * da + \log(a) * a ** b * db$$

$$c = \cos(a) \quad \Rightarrow \quad dc = -\sin(a) * da$$

$$c = \tan(a) \quad \Rightarrow \quad dc = da / \cos(a) ** 2$$

Forward-mode AD

To translate using the rules we simply replace each primitive operation in the original program by its differential analogue.

The order of computation remains unchanged: if a statement K is evaluated before another statement L, then the differential analogue of K is evaluated before the analogue statement of L.

Interleaving differential computation

A careful analysis of our original program and its differential analogue shows that its possible to interleave the differential calculations with the original ones:

`x = ?`

`dx = ?`

`y = ?`

`dy = ?`

`a = x * y`

`da = y * dx + x * dy`

`b = sin(x)`

`db = cos(x) * dx`

`z = a + b`

`dz = da + db`

- This implies that we can keep track of the value and gradient at the same time.
- We can use a mathematical concept called a “Dual Number” to create a very simple direct implementation of AD.

Forward-mode AD

- **Advantage:** easy to implement,
- **Big Disadvantage:**

For every variable we wish to compute the gradient with respect to, we have to run the complete program again.

This is obviously going to be a problem if we're talking about the gradients of a function with very many parameters (e.g. a deep network).

Reversing the chain rule

The **chain rule is symmetric** — this means we can turn the derivatives upside-down:

$$\frac{\partial s}{\partial u} = \sum_{i=1}^N \frac{\partial s}{\partial w_i} \frac{\partial w_i}{\partial u} = \frac{\partial s}{\partial w_1} \frac{\partial w_1}{\partial u} + \dots + \frac{\partial s}{\partial w_N} \frac{\partial w_N}{\partial u}$$

In doing so, we have inverted the input-output role of the variables:

u is some input variable,

w_i 's are the output variables that depend on u ,

s is the yet-to-be-given variable.

In this form, the chain rule can be applied repeatedly to every input variable u (akin to how in forward mode we repeatedly applied it to every w).

Therefore, given some s we expect this form of the rule to give us a program to compute both $\frac{\partial s}{\partial x}$ and $\frac{\partial s}{\partial y}$ in one go ...

Reversing the chain rule — Example

$$\frac{\partial s}{\partial u} = \sum_{i=1}^N \frac{\partial s}{\partial w_i} \frac{\partial w_i}{\partial u}$$

$x = ?$

$y = ?$

$a = x y$

$b = \sin(x)$

$z = a + b$

$$\frac{\partial s}{\partial z} = ?$$

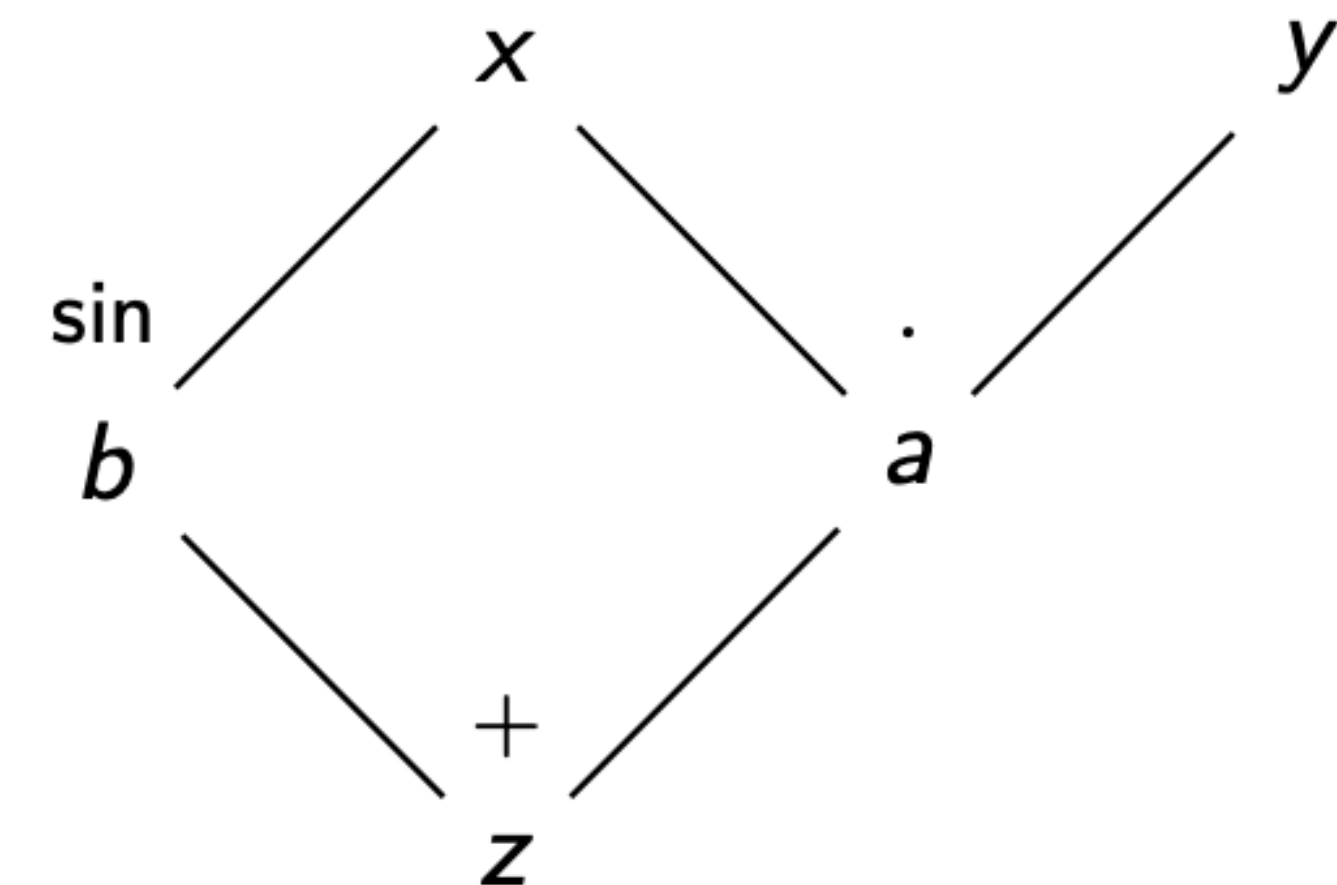
$$\frac{\partial s}{\partial b} = \frac{\partial z}{\partial b} \frac{\partial s}{\partial z} = \frac{\partial s}{\partial z}$$

$$\frac{\partial s}{\partial a} = \frac{\partial z}{\partial a} \frac{\partial s}{\partial z} = \frac{\partial s}{\partial z}$$

$$\frac{\partial s}{\partial y} = \frac{\partial a}{\partial y} \frac{\partial s}{\partial a} = x \frac{\partial s}{\partial a}$$

$$\begin{aligned} \frac{\partial s}{\partial x} &= \frac{\partial a}{\partial x} \frac{\partial s}{\partial a} + \frac{\partial b}{\partial x} \frac{\partial s}{\partial b} \\ &= y \frac{\partial s}{\partial a} + \cos(x) \frac{\partial s}{\partial b} \\ &= (y + \cos(x)) \frac{\partial s}{\partial z} \end{aligned}$$

Differentiating in reverse can be quite mind-bending: instead of asking what input variables an output depends on, we have to ask what output variables a given input variable can affect. We can see this visually by drawing a dependency graph of the expression:



visualize dependencies

Translating to code

Let's now translate our derivatives into code.

As before we replace the derivatives ($\partial s / \partial z$, $\partial s / \partial b$, ...) with variables (g_z , g_b , ...) which we call **adjoint variables**:

```
gz = ?  
gb = gz  
ga = gz  
gy = x * ga  
gx = y * ga + cos(x) * gb
```

If we go back to the equations and substitute $s = z$ we would obtain the gradient in the last two equations.

In the above program, this is equivalent to setting $g_z = 1$.

This means to get the both gradients $\partial z / \partial x$ and $\partial z / \partial y$ we only need to run the program once!

Limitations of Reverse Mode AD

If we have multiple output variables, we would have to run the program for each one (with different seeds on the output variables) .

For example:

$$\begin{cases} z = 2x + \sin x \\ v = 4x + \cos x \end{cases}$$

We can't just interleave the derivative calculations (since they all appear to be in reverse)...

How can we make this automatic?

Implementing Reverse Mode AD

There are two ways to implement Reverse AD:

1. We can parse the original program and generate the adjoint program that calculates the derivatives.
 - Potentially hard to do.
 - Static, so can only be used to differentiate algorithms that have parameters predefined.
 - But, efficient (lots of opportunities for optimisation)
2. We can make a dynamic implementation by constructing a graph that represents the original expression as the program runs.

Constructing an expression graph

The goal is to get something akin to the graph we saw earlier:

The “roots” of the graph are the independent variables x and y .

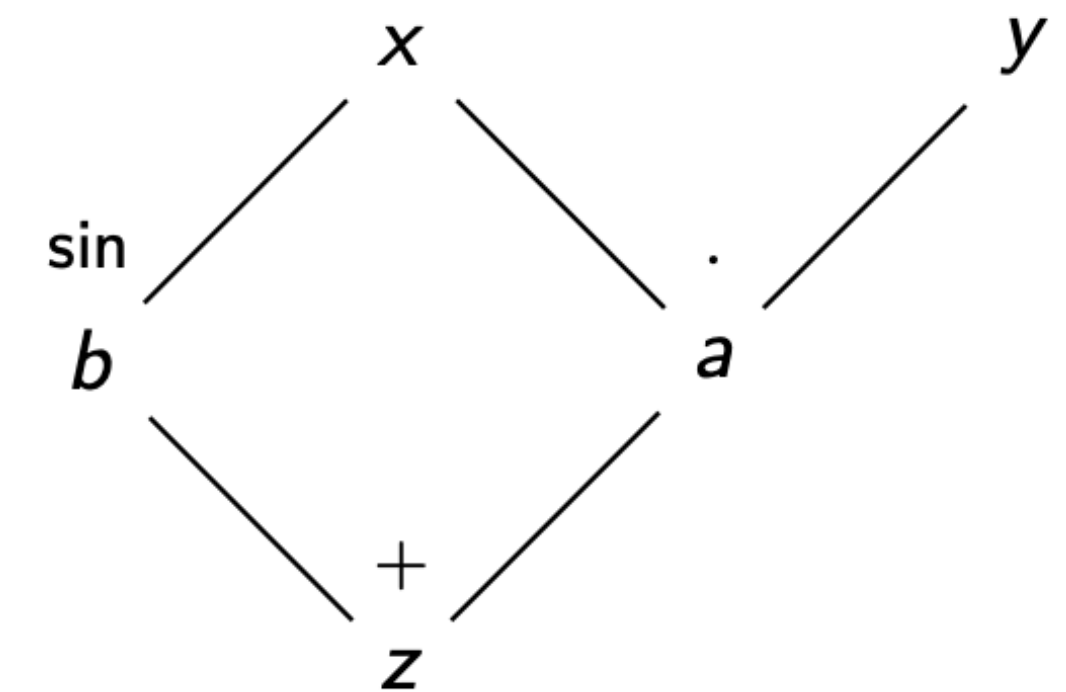
Constructing these nodes is as simple as creating an object:

```
class Var:
    def __init__(self, value):
        self.value = value
        self.children = []
        ...
    ...
```

```
x = Var(0.5)
y = Var(4.2)
```

Each Var node can have children which are the nodes that depend directly on that node.

The children allow nodes to link together in a **Directed Acyclic Graph**.



visualize dependencies

Building expressions

By default, nodes do not have any children. As expressions are created each expression u registers itself as a child of each of its dependencies w_i together with its weight $\partial w_i / \partial u$ which will be used to compute gradients.

```
class Var:
    ...
    def __mul__(self, other):
        z = Var(self.value * other.value)

        # weight = dz/dself = other.value
        self.children.append((other.value, z))

        # weight = dz/dother = self.value
        other.children.append((self.value, z))
        return z
    ...
    ...
    # "a" is a new Var that is a child of both x and y
    a = x * y
```

Building expressions

By default, nodes do not have any children. As expressions are created each expression u registers itself as a child of each of its dependencies w_i together with its weight $\partial w_i / \partial u$ which will be used to compute gradients.

```
class Var:
    ...
    def __mul__(self, other):
        z = Var(self.value * other.value)

        # weight = dz/dself = other.value
        self.children.append((other.value, z))

        # weight = dz/dother = self.value
        other.children.append((self.value, z))
        return z
    ...
    ...
    # "a" is a new Var that is a child of both x and y
    a = x * y
```

```
x = Var(0.5)
```

```
y = Var(4.2)
```

```
a = x.mul(y)
```

```
z = Var(0.5*4.2)
```

```
x.children((y.value, z))
```

```
y.children((x.value, z))
```

```
a ← z
```

Weight $\partial z / \partial x = y$

no childrens for z

z depends
on x and y

Weight $\partial z / \partial y = x$

Computing gradients

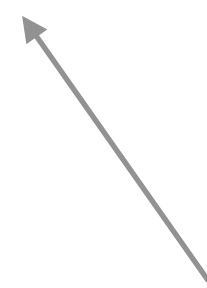
Finally, to get the gradients we need to propagate the derivatives. To avoid unnecessarily traversing the tree multiple times we will cache the derivative of a node in an attribute `grad_value`:

```
class Var:
    def __init__(self):
        ...
        self.grad_value = None

    def grad(self):
        if self.grad_value is None:
            # calculate derivative using chain rule
            self.grad_value = sum(weight * var.grad() for weight,
                                   var in self.children)
        return self.grad_value
    ...

...
a.grad_value = 1.0
print("da/dx_{}={}".format(x.grad()))
```

dynamic programming



AD in the PyTorch Autograd package

- PyTorch's AD is remarkably similar to the one we've just built:
 - it builds the computation graph as it runs (recording explicit Function objects as the children of Tensors rather than grouping everything into Var objects)
 - it caches the gradients in the same way we do (in the grad attribute) - hence the need to call `zero_grad()` when recomputing the gradients of the same graph after a round of backprop.
- PyTorch does some clever memory management to work well in a reference-counted regime and aggressively frees values that are no longer needed.
- The backend is actually mostly written in C++, so it's fast, and can be multi-threaded.
- It allows easy "turning off" of gradient computations through `requires_grad`.
- In-place operations which invalidate data needed to compute derivatives will cause runtime errors, as will variable aliasing...

Careful with matrix multiplications..

$$ABx = (AB)x = A(Bx)$$

$O(n^3)$ computations
 $O(n^2)$ memory

$O(n^2)$ computations
 $O(n)$ memory

Same for $uAB = (uA)B = u(AB)$

Derivatives and gradients

Derivative of $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ as Jacobian $\frac{\partial f}{\partial x}$, i.e. matrix of partial derivatives of components of f .

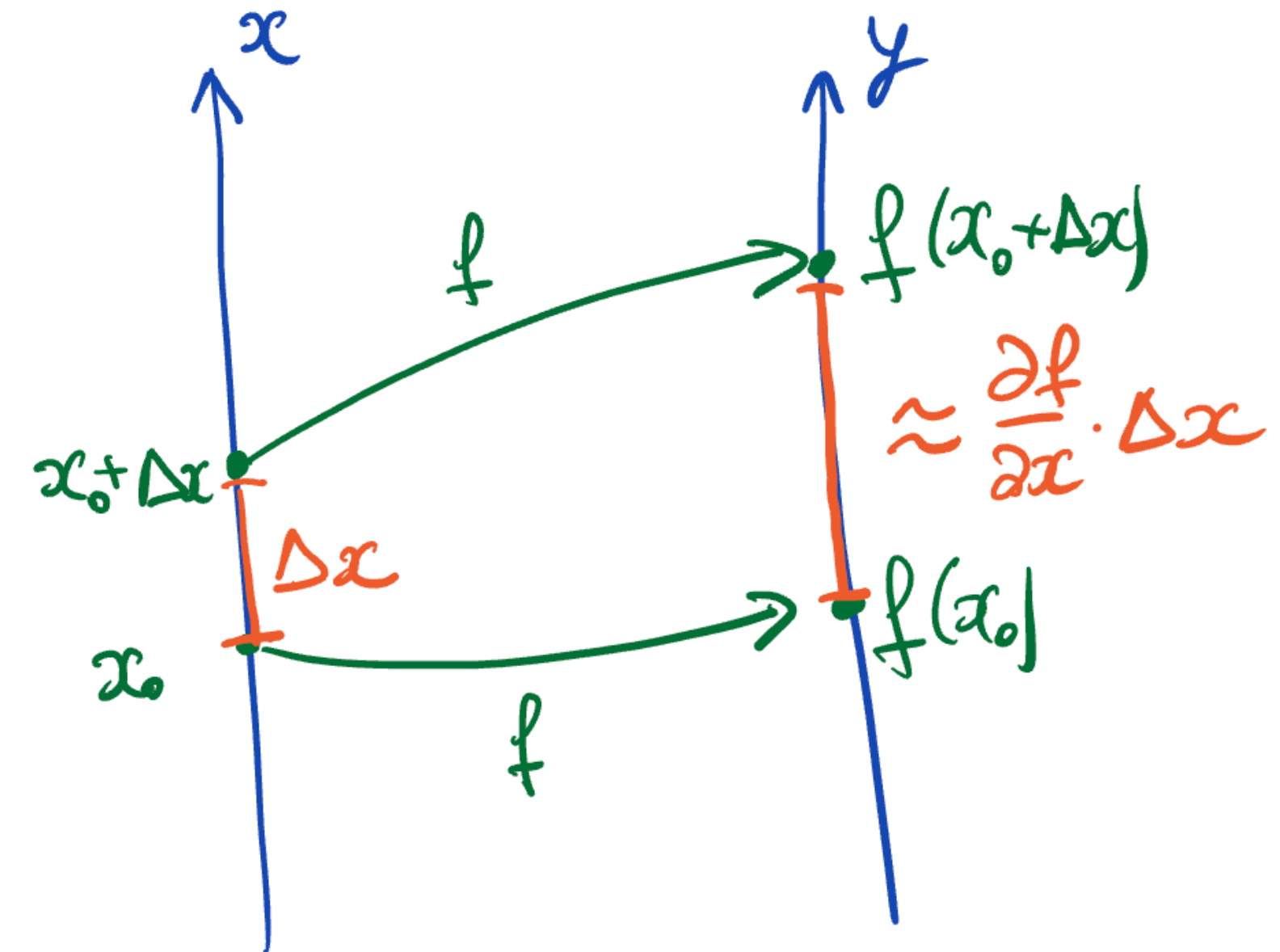
$\frac{\partial f}{\partial x}$ is a matrix has n columns and m rows.

By definition of a derivative, for any vector $\Delta x \in \mathbb{R}^n$,

$$f(x + \Delta x) - f(x) = \frac{\partial f(x)}{\partial x} \Delta x + o(\|\Delta x\|)$$

or informally

$$f(x + \Delta x) - f(x) \approx \frac{\partial f(x)}{\partial x} \Delta x$$

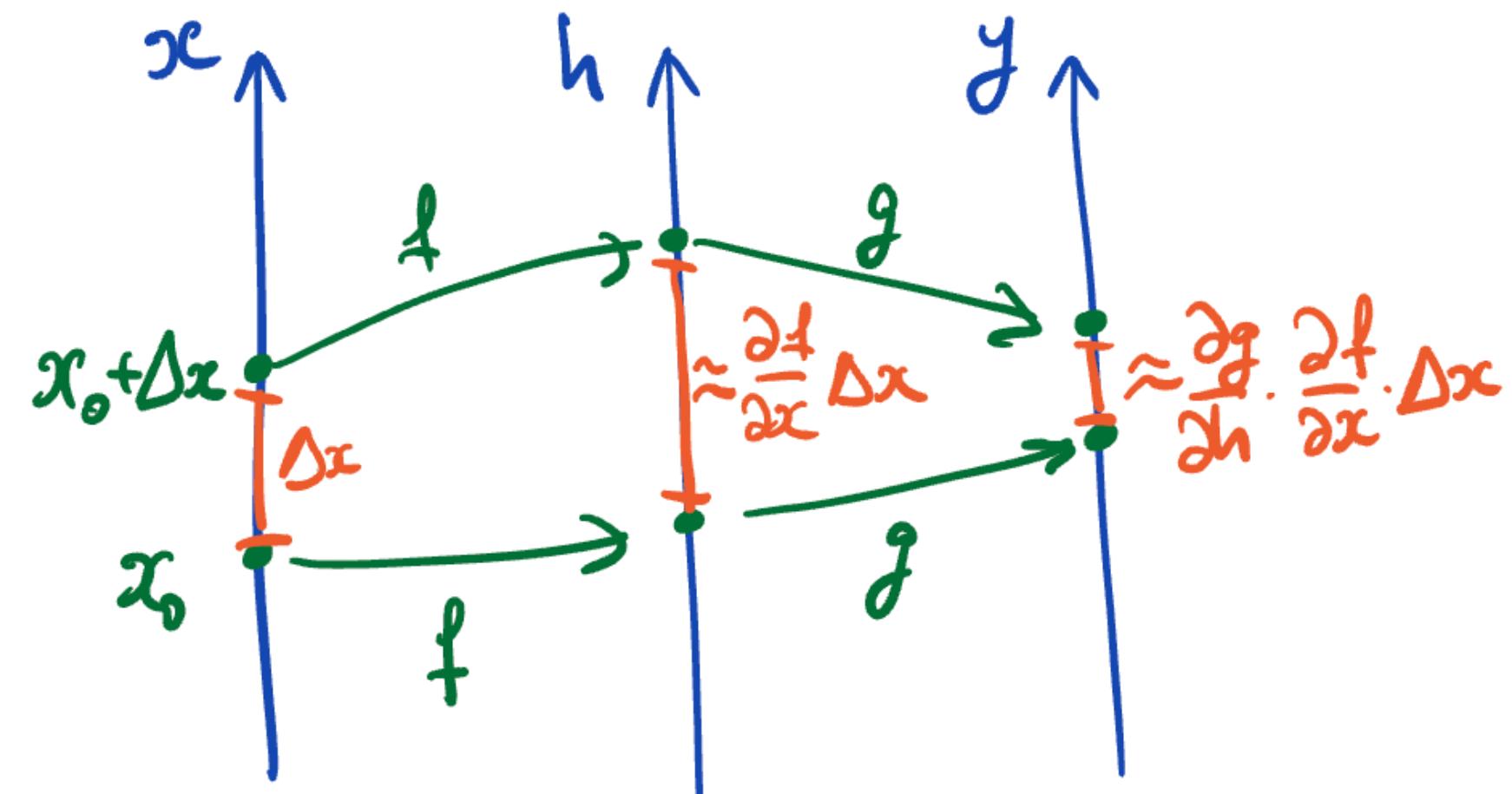


the length of an image of the small segment is approximately equal to the derivative multiplied by the length of the segment itself

Chain Rule

$$\frac{\partial(g \circ f(x))}{\partial x} = \frac{\partial g(h)}{\partial h} \frac{\partial f(x)}{\partial x}$$

where the first derivative is taken at point $h = f(x)$



Chain rule

Gradient of composition

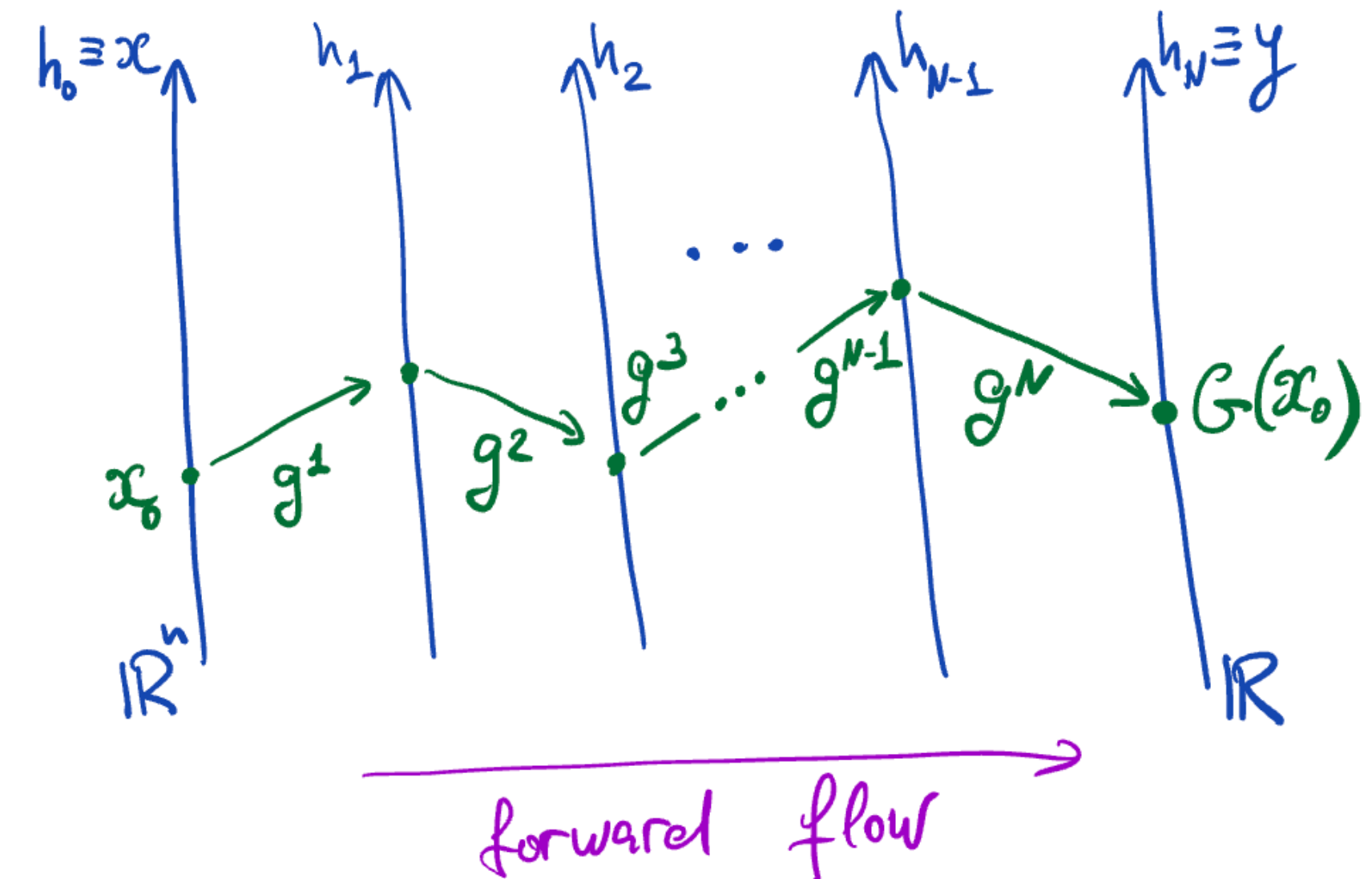
$$G : \mathbb{R}^n \rightarrow \mathbb{R}$$

$$G(x) = g^N \circ g^{N-1} \circ \dots \circ g^1(x)$$

where $g^1, \dots, g^N$ are some differentiable maps from multidimensional spaces to multidimensional spaces.

Forward pass

$$h_1 := g^1(x), h_2 := g^2(h_1), \dots, y = g^N(h_{N-1})$$



Gradient of composition

Chain rule $\nabla_x G = \frac{\partial g^N}{\partial h_{N-1}} \frac{\partial g^{N-1}}{\partial h_{N-2}} \dots \frac{\partial g^2}{\partial h_1} \frac{\partial g^1}{\partial x}$

g^N maps to $\mathbb{R}$ so:



$$\nabla_x G = \nabla_{h_{N-1}} g^N \frac{\partial g^{N-1}}{\partial h_{N-2}} \dots \frac{\partial g^2}{\partial h_1} \frac{\partial g^1}{\partial x}$$

How to use this equation to find the gradient?

First of all, we have to find $h_1, \dots, h_{N-1}$, i.e. perform all the steps of the forward pass (except the last one).

Then we have to find all the derivatives of functions $g^1, \dots, g^N$ at the corresponding points $h_{N-1}, h_{N-2}, \dots, h_1, x$

Then we have to multiply everything (the most natural and efficient way is to do it **left-to-right**)

$$\nabla_{h_{N-1}} g^N \frac{\partial g^{N-1}}{\partial h_{N-2}}$$

Backward pass: calculation flow goes backward, from the terms with large indexes to the terms with small indexes

Neural Networks

Loss function $\mathcal{L}(\theta) := L(f_{\theta}(x_{input}), y_{true})$

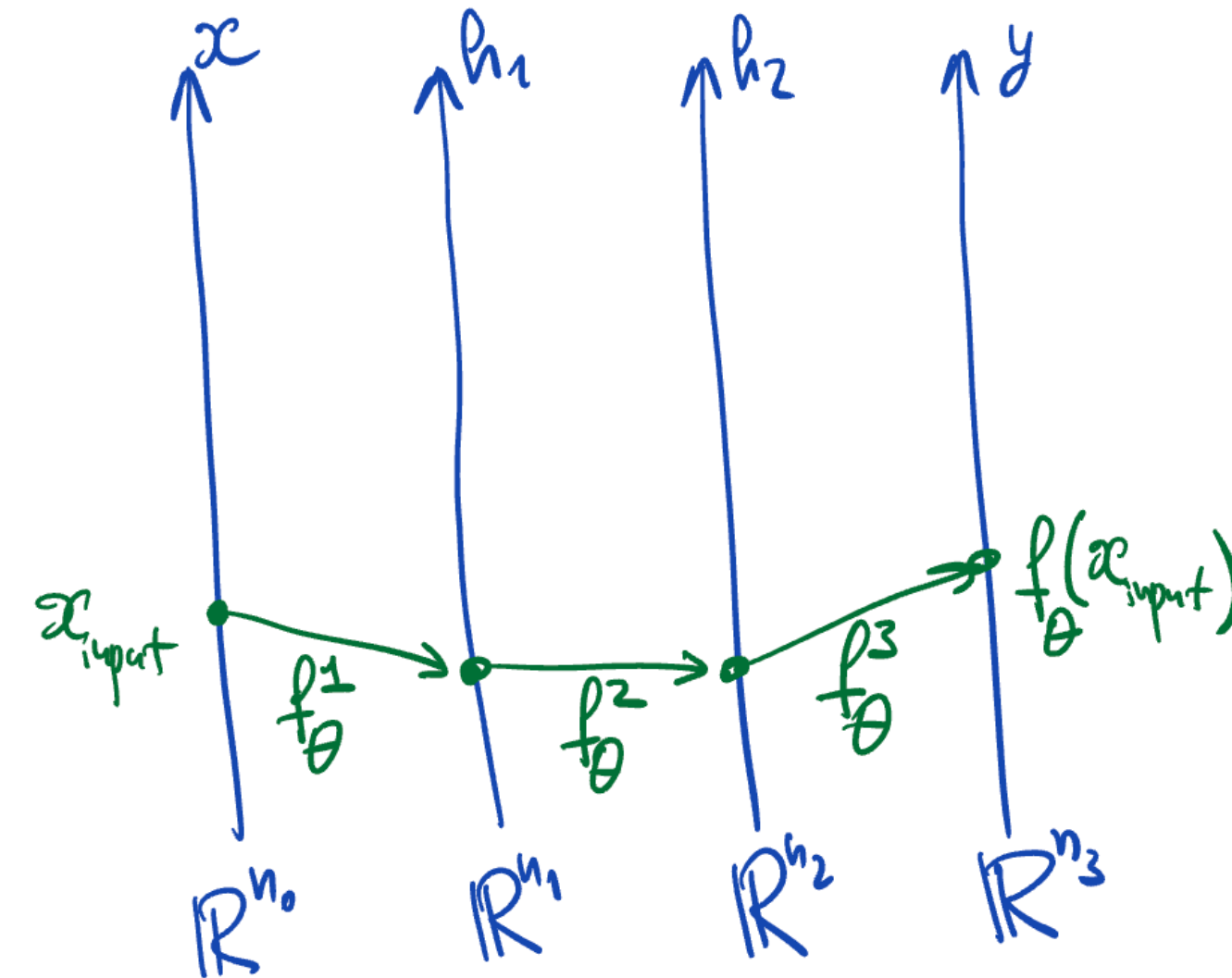
$$\nabla_{\theta} \mathcal{L}(\theta) = \nabla_y L \cdot \frac{\partial f_{\theta}(x_{input})}{\partial \theta}$$

Easy to find $\nabla_y L$, provided that y is already calculated

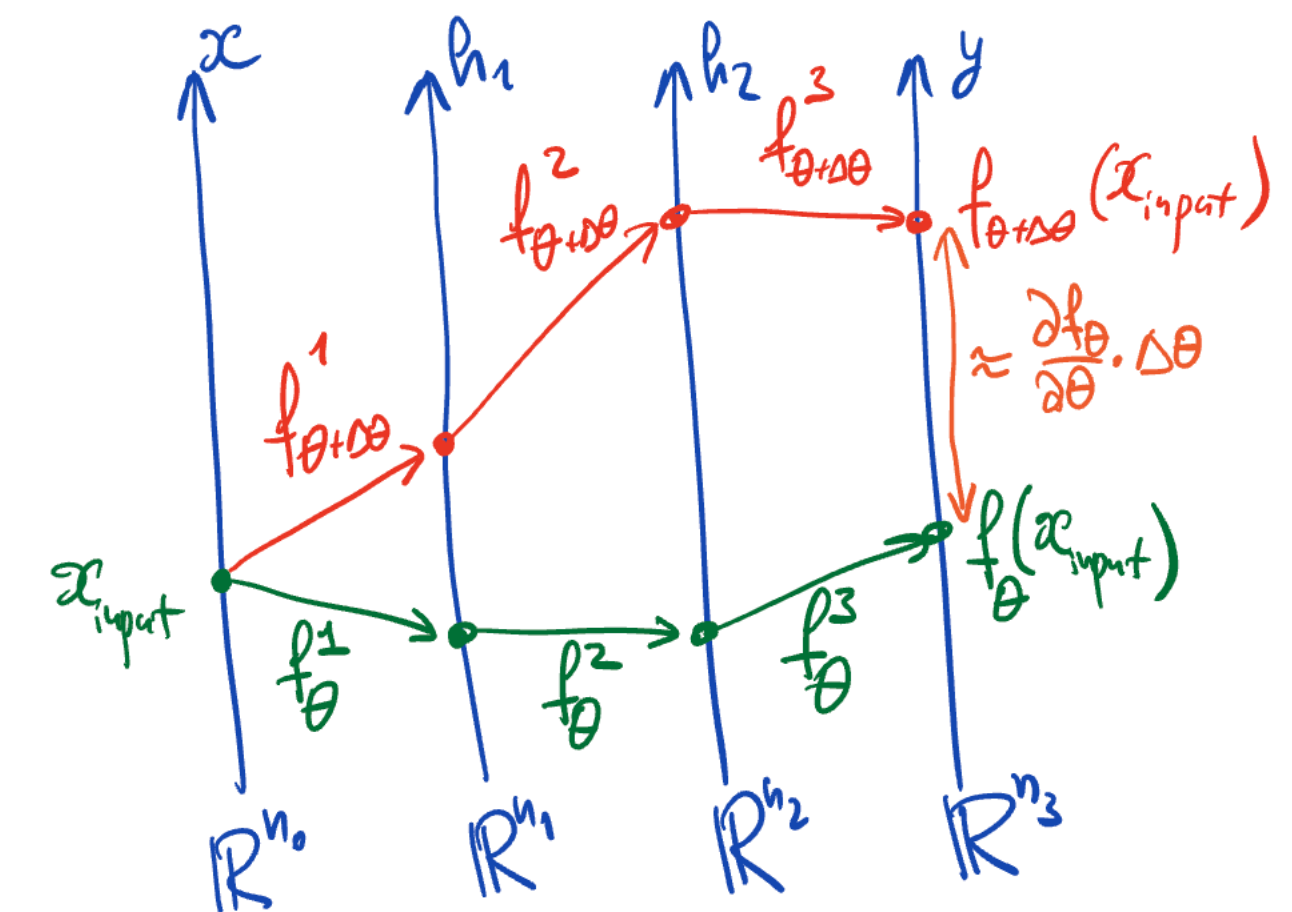
E.g. quadratic loss $L(y, y_{true}) = (y - y_{true})^2 \rightarrow \nabla_y L = 2y - 2y_{true}$

Each function now depends not only on its argument but also on the parameter θ

The difference between outputs $f_{\theta+\Delta\theta}(x_{input}) - f_{\theta}(x_{input}) \approx \frac{\partial f_{\theta}}{\partial \theta} \Delta\theta$



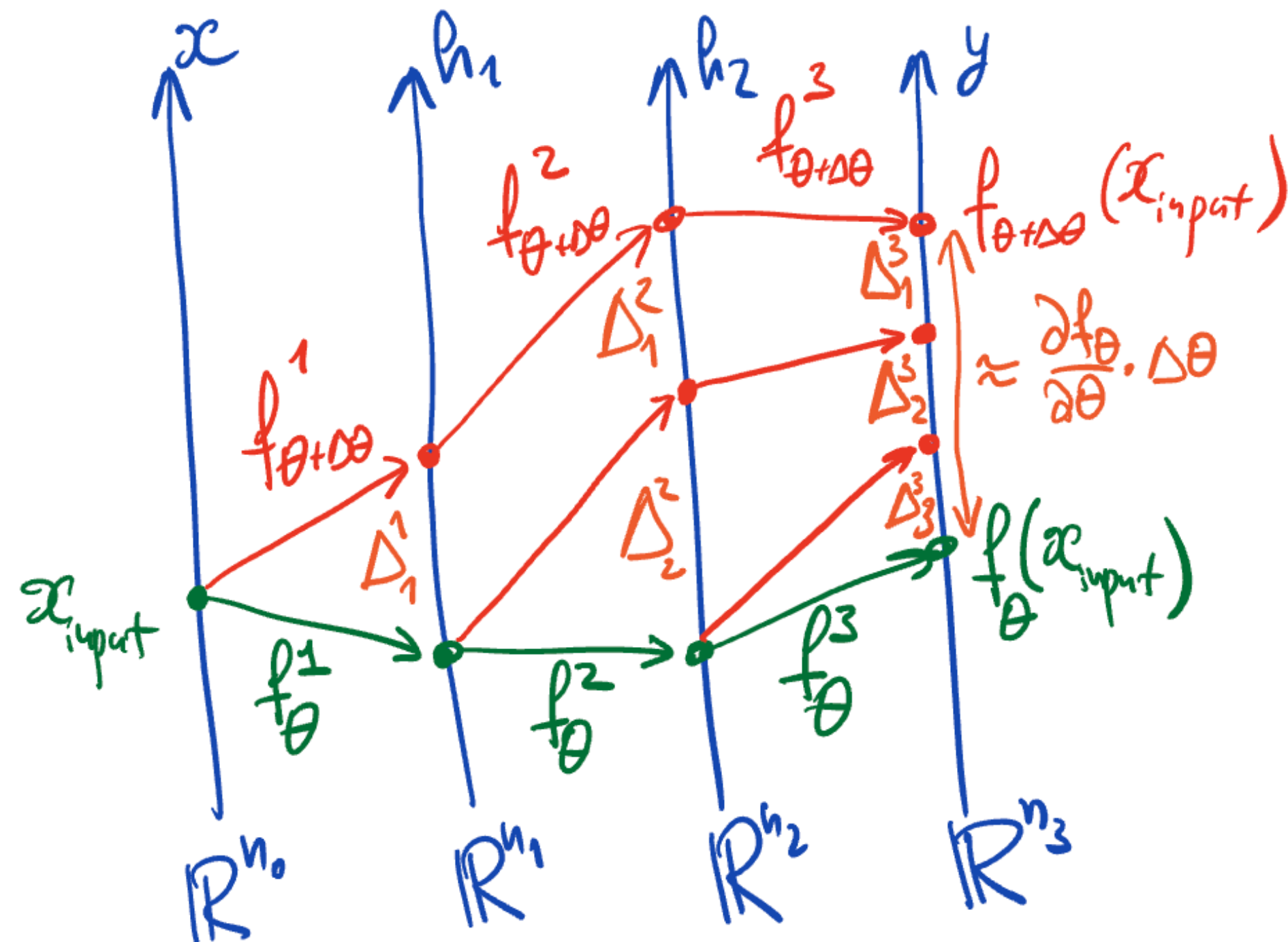
Three-layer neural network



What happens with the output of neural network if we slightly change parameters.

Derivative of the Network

For each of the intermediate points of the unperturbed trajectory, $f_\theta^1(x_{input}), f_\theta^2 \circ f_\theta^1(x_{input})$, we consider a trajectory of the perturbed network that starts from this point. These trajectories split the segment $[f_\theta(x_{input}), f_{\theta+\Delta\theta}(x_{input})]$ into three smaller segments denoted (from top to bottom) by $\Delta_1^3, \Delta_2^3, \Delta_3^3$



Decomposition of the network's derivative

$\Delta_3^3 \approx \frac{\partial f_\theta^3}{\partial \theta} \Delta \theta \rightarrow$ measures the difference between the images of some point under action of $f_{\theta+\Delta\theta}^3$ and f_θ^3

Consider Δ_2^3

$\Delta_2^2 \approx \frac{\partial f_\theta^2}{\partial \theta} \Delta \theta \rightarrow f_{\theta+\Delta\theta}^2$ and f_θ^2 applied at the same point

$\Delta_2^3 \approx \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \Delta_2^2 \approx \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \frac{\partial f_\theta^2}{\partial \theta} \Delta \theta \rightarrow f_{\theta+\Delta\theta}^2$ is applied to two different points. To find the difference between the images now, we have to use the derivative of $f_{\theta+\Delta\theta}^3$ with respect to its argument h_2

Derivative of the Network

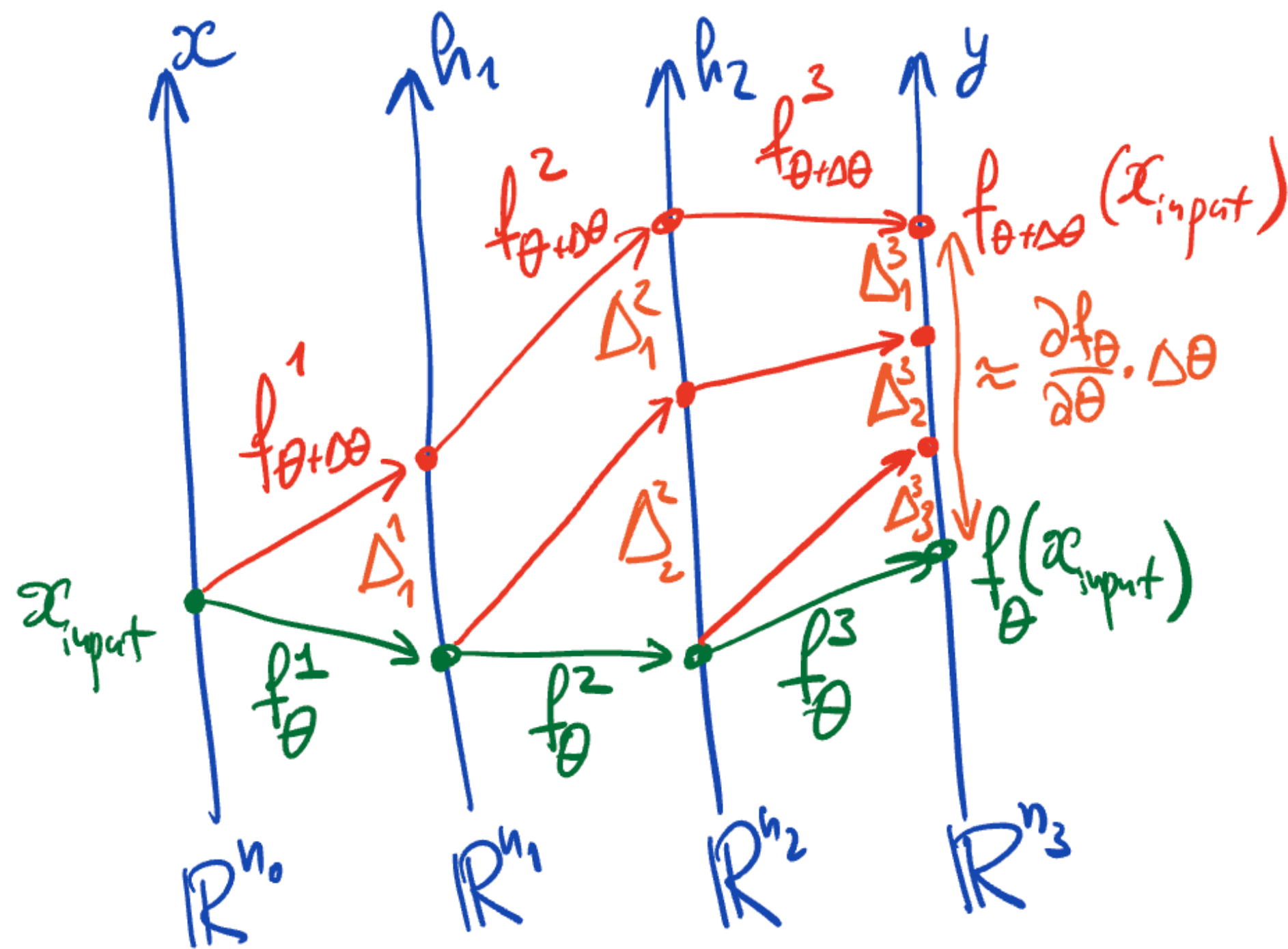
$$\Delta_1^3 \approx \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \Delta_1^2 \approx \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \frac{\partial f_{\theta+\Delta\theta}^2}{\partial h_1} \Delta_1^1 \approx \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \frac{\partial f_{\theta+\Delta\theta}^2}{\partial h_1} \frac{\partial f_{\theta}^1}{\partial \theta} \Delta\theta.$$

Now let's sum up everything:

$$\begin{aligned} \frac{\partial f_{\theta}}{\partial \theta} \Delta\theta \approx \Delta_3^3 + \Delta_2^3 + \Delta_1^3 \approx \\ \left(\frac{\partial f_{\theta}^3}{\partial \theta} + \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \frac{\partial f_{\theta}^2}{\partial \theta} + \frac{\partial f_{\theta+\Delta\theta}^3}{\partial h_2} \frac{\partial f_{\theta+\Delta\theta}^2}{\partial h_1} \frac{\partial f_{\theta}^1}{\partial \theta} \right) \Delta\theta. \end{aligned}$$

As $\Delta\theta$ tends to zero, the approximations become more and more precise, and now one can easily believe that

$$\frac{\partial f_{\theta}}{\partial \theta} = \frac{\partial f_{\theta}^3}{\partial \theta} + \frac{\partial f_{\theta}^3}{\partial h_2} \frac{\partial f_{\theta}^2}{\partial \theta} + \frac{\partial f_{\theta}^3}{\partial h_2} \frac{\partial f_{\theta}^2}{\partial h_1} \frac{\partial f_{\theta}^1}{\partial \theta}.$$



Decomposition of the network's derivative

Coming next...

Physics-Informed ML

- Neural ODEs